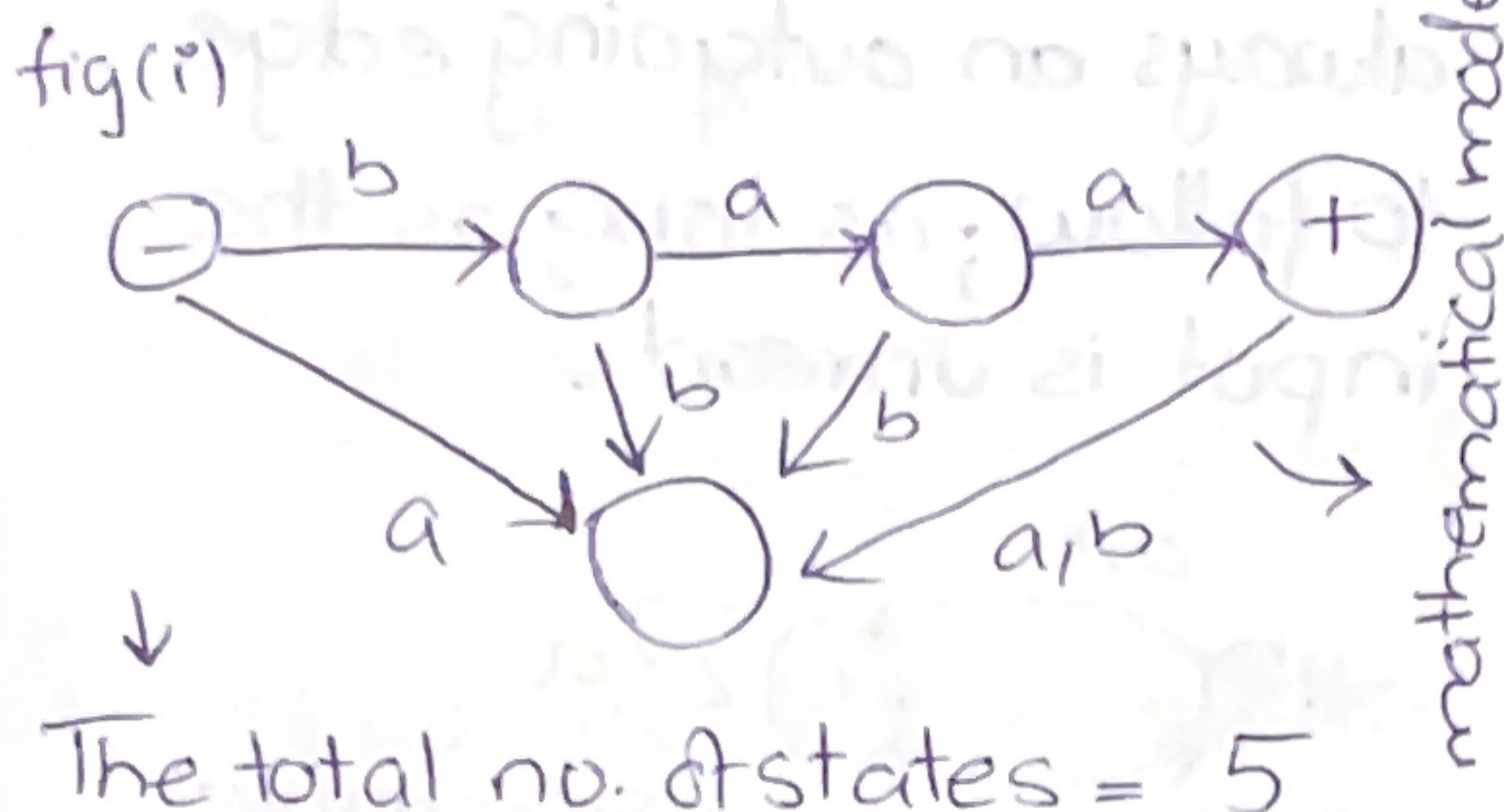


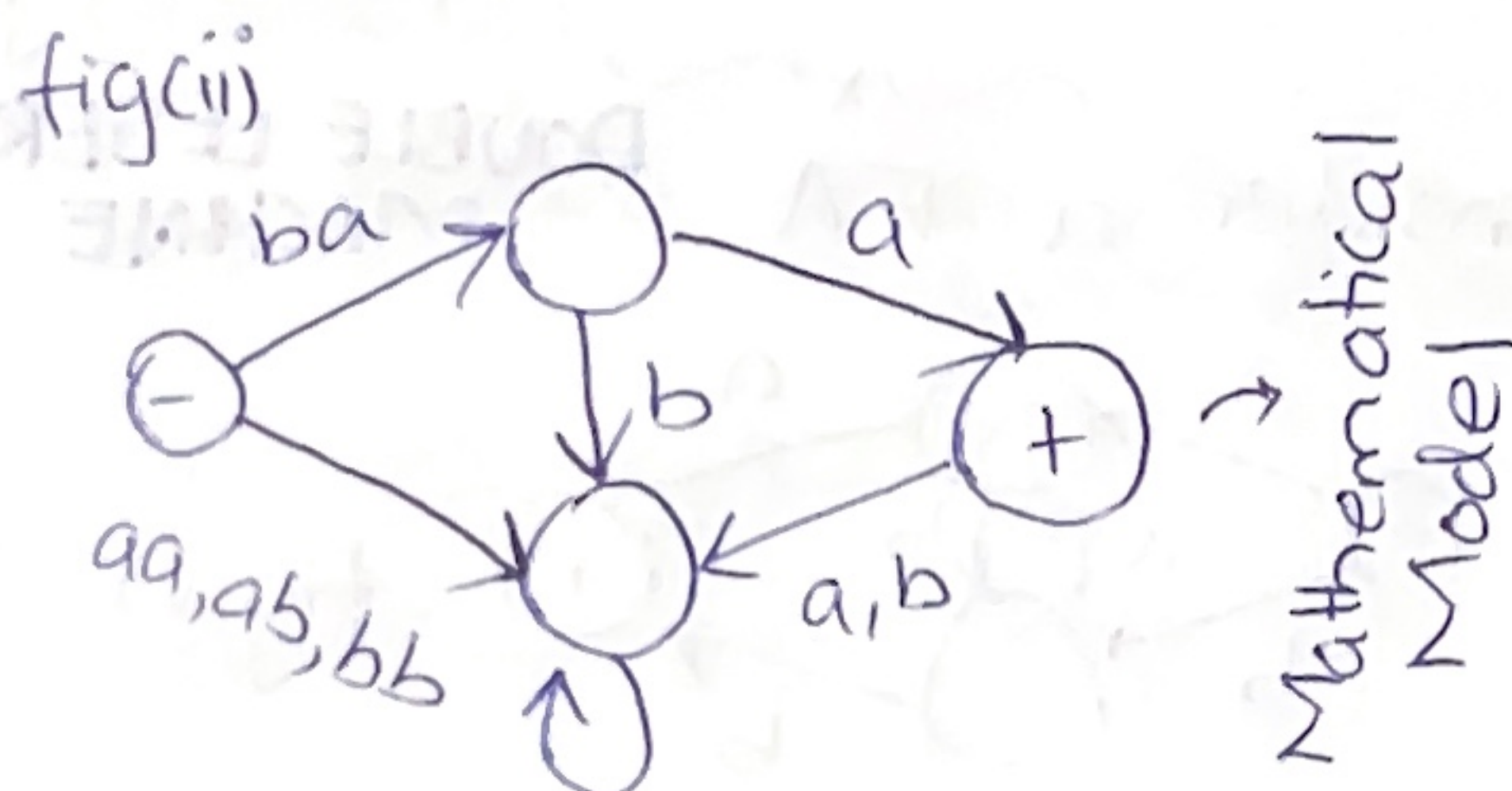
Transition Graphs.

Previously we saw an FA that accepts the string 'baa' as,



Why? Because FA can read one letter from the input at a time.

Now; let us consider a more powerful machine that can read one or two letter at a time.

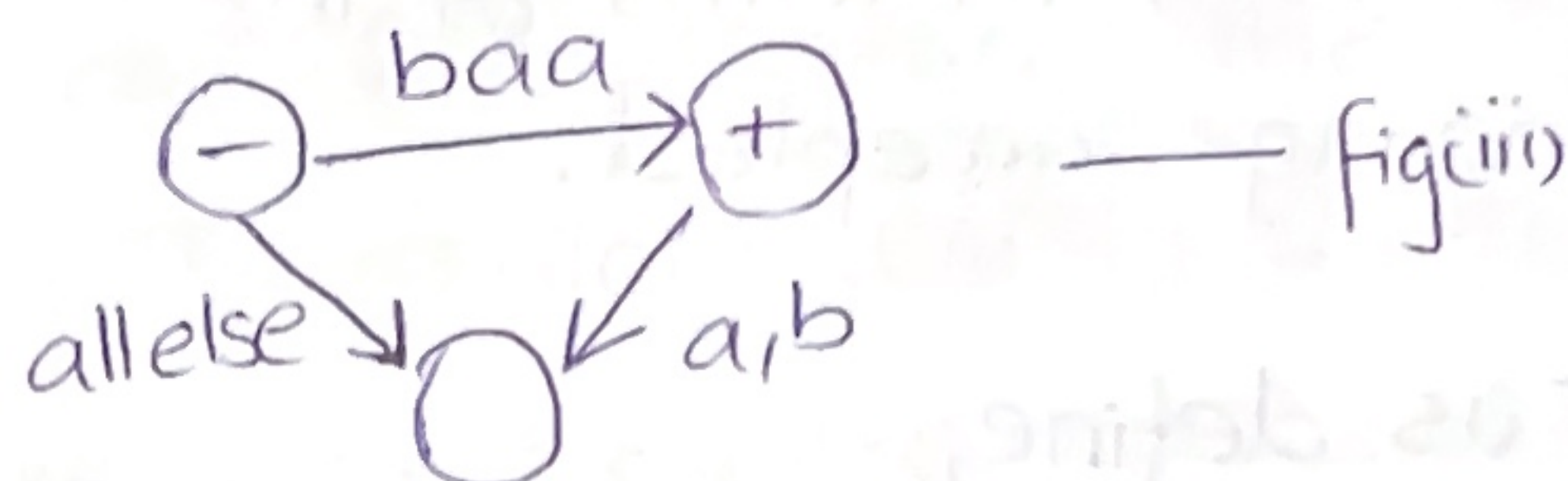


This model differs in a significant way than the previous implementation at fig (i).

The states and edges still exist but the requirement to read only one letter is abandoned!

Question If a machine M only accepts the input string 'baa' than why only read 2 letters at a time?

rather; we can also do



OR



In the model made at fig (iv) all the other strings end nowhere.

- let say, we start at 0 in fig (iv) and read 'a'; where to go then?

In the model made at fig (iii) at least we know where to go other than 'baa'

Similarly

let's consider 'baabbb' as input for model at fig (iv)

- first three states takes to (+)
- where to go after that when bb is read?

→ P.T.O
continued....

NOTE:- FA says that we cannot until string ends.

- By convention, we can thus say that it is there but not shown.

fig(iii) & fig(iv) models are equivalent in terms of the language accepted.

LET us define,

What happens when an input running on a machine gets into a state from which it cannot escape though it has not been fully read?

Definition

When an input string that has not been completely read reaches a state (either final or otherwise) where it cannot leave because there is no outgoing edge to follow;

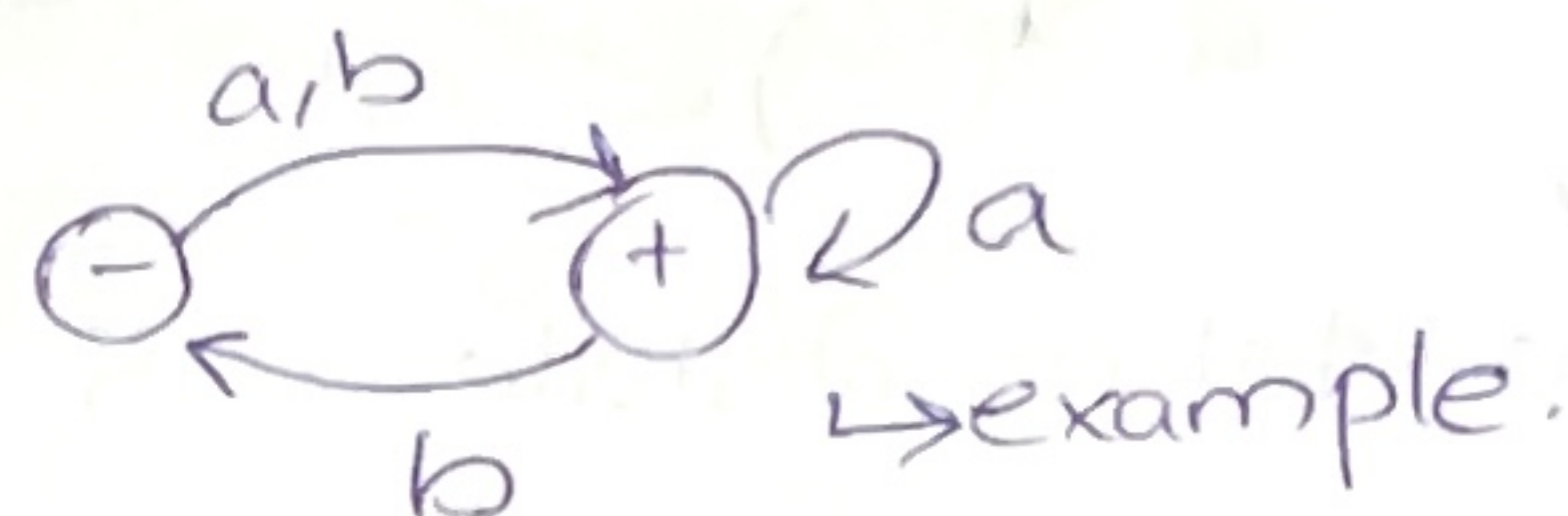
then say that input/machine CRASHES at that state; i.e:

- (i) Execution terminates
- (ii) Input is rejected.

Note:-

In FA no input can crash.

Why? Because there is always an outgoing edge to follow; as long as the input is unread.



Now

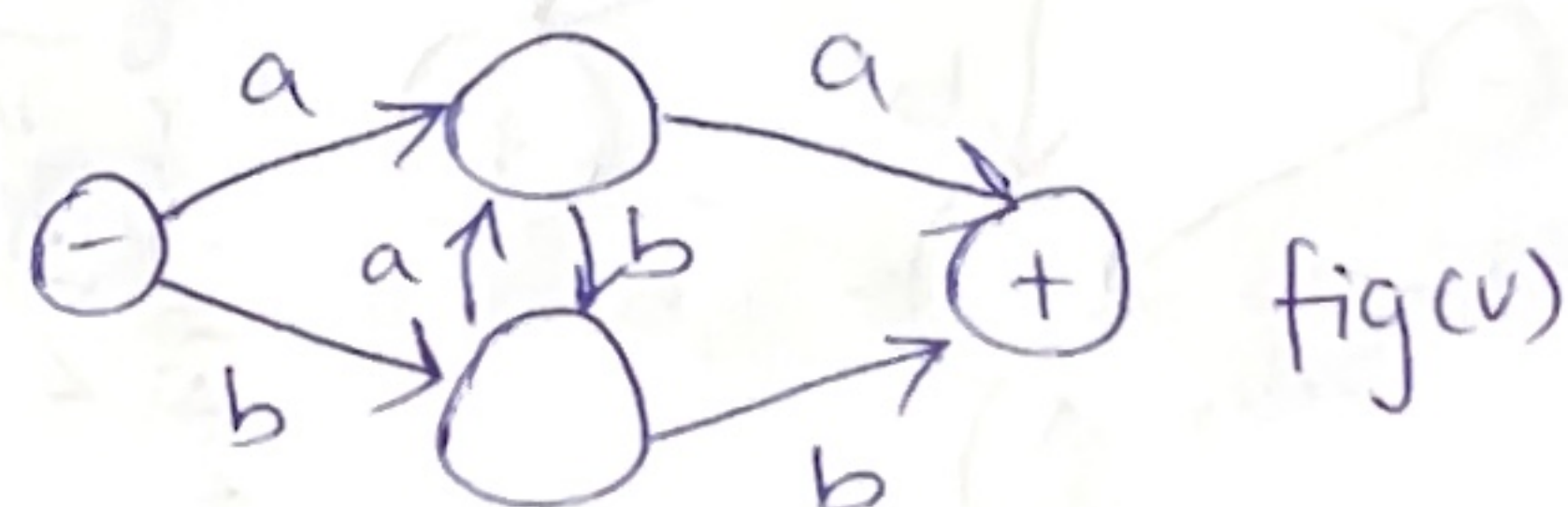
we have to decide about how the input is rejected?

There are 2 possibilities.

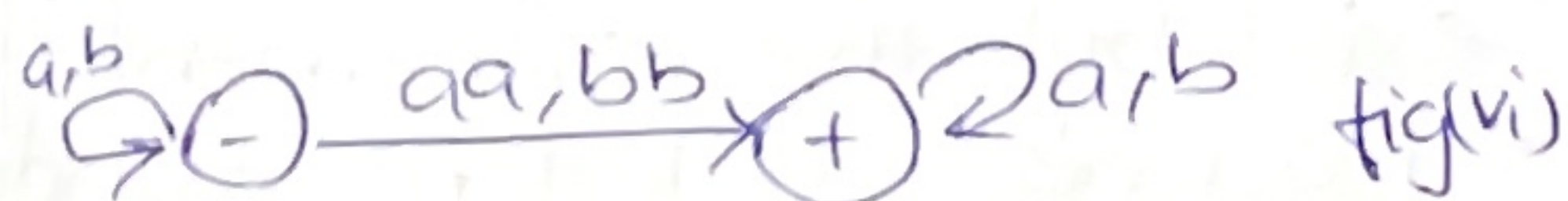
① Input does not end in the final state.

② Crashed while processing

Consider a FA ^{DOUBLE LETTER MACHINE}



If we say that the Machine in fig(v) can read two or (one letter) at a time then we can build the machine using 2 states to accept the double letter as:



The machine in fig(vi) we changed not only;
 - the way edges are labelled and no. of letters to read at a time

But, we also changed how many letters should be read each time we go back for more.

~x~

Consider the input string 'baa' (isn't that easy to read) How this machine will accept this string?

(i) start with b

(ii) Reads aa

However it; reads b and stays at

⊖

(2) Decides whether to read another single character a.

(3) Again read 'a' and we are still at initial state.

can we accept this string?

Fo'd 2 different paths for

'baa'

- (i) one leads to acceptance
- (ii) Other " " rejection

Another way to look

(i) start ba

- its not double

- do we have any edge to follow?

(we cannot move/ find, why?)

because 'ba' is not having edge from start state.

hence processing breaks and machine crashes.

∴ Input strings can be read by variable amount/no. of letters.

Input string can follow variety of paths through the machine.

What should be the conclusion

Is string part of the language?

Not Clear

It must be an absolute

Yes or No answer or

else wise the language is not well defined.

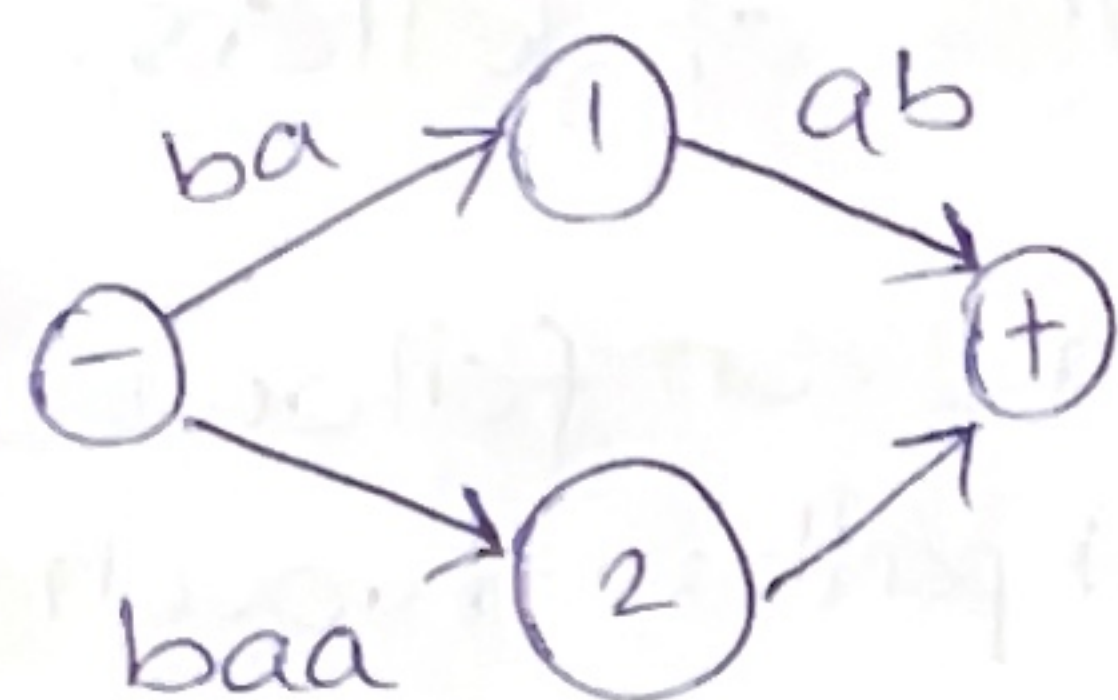
Result; if we want the machine to accept more than 1 letter we must change 'Definition of Acceptance'.

we can say that,

- a string is accepted if it can be processed (so as to arrive at the final state / $\oplus$).
- Ignore Failures
i.e do not lead to $\oplus$

$\therefore$ Now we are about to introduce a machine which can be labelled by any string of alphabets.

Consider



Problem

baab $\rightarrow$ acceptance?
is in two ways



Previously ⁱⁿ FA;

- we had unique paths.
- Now some strings have no path at all and some may have many paths...

Conclusion:

As we have observed many difficulties while expanding machine

i.e to allow to read more than one letters at a time

So now let's define a new machine called as the

TRANSITION GRAPHS $\rightarrow$ TGs

because they are well understood directly as graphs then tables and then turned into pictures.

DEFINITION:-

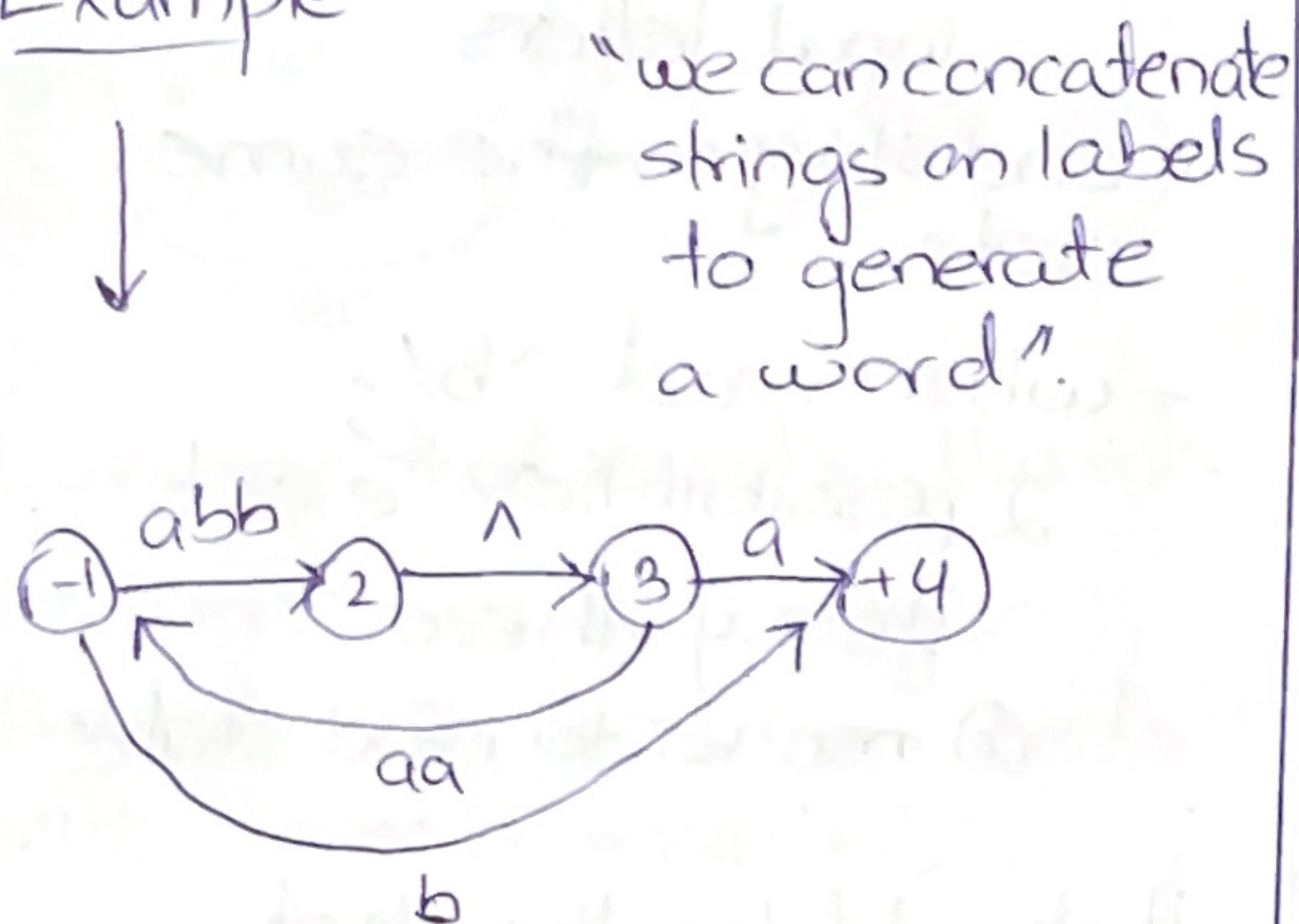
- ① A set of finite states at least one of which is designated as the $\ominus$ initial/start state and some (or maybe none) of which are designated as $\oplus$
- ② A Σ would be defined.
- ③ Finite set of transitions that show how to go from some states to some others, based on reading a specified string (possibly even the Λ).

- ⇒ (3) meant also that the edges can be labelled by some strings of letters, not necessarily one letter.

Also requiring that there be any specific no. of edges emitting from state.

→ some states may have no edges and some may have many.

Example



Path: ① — ② — ③ — ① — ④

→ strings (abb)(^)(aa)(b)

for the word abbaab.

some other strings

'abba', 'abbaaaba' and 'b'

* Edge labelled as '^' means no consumption of letters

Remember

- ∴ edge labelled '^' do not need to follow a path but we can take it if we want to.

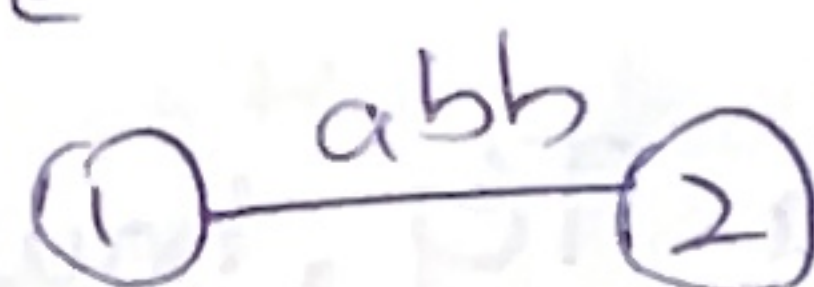
Note: you must break the given string into substrings that might correspond to labels of edges on path.

Example: abbaab.

From states can take

abb or b.

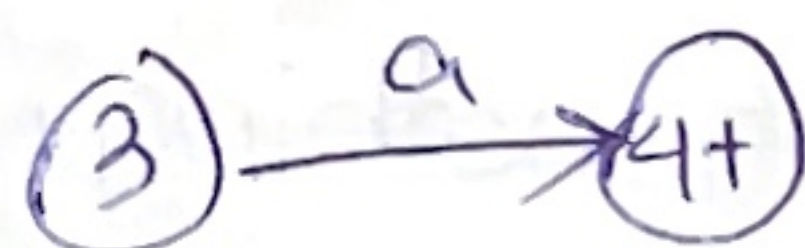
for this 'abb' or 'b' we move



left ab, current state is ②



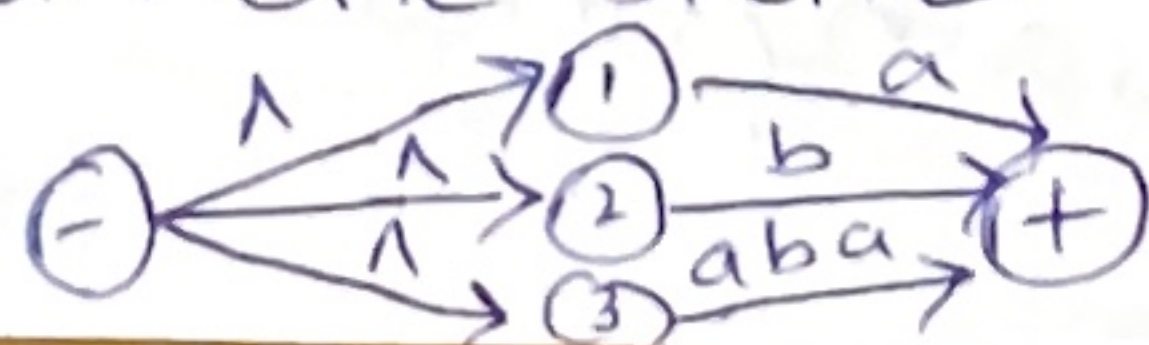
At 3 we cannot read ab so must read 'a'.

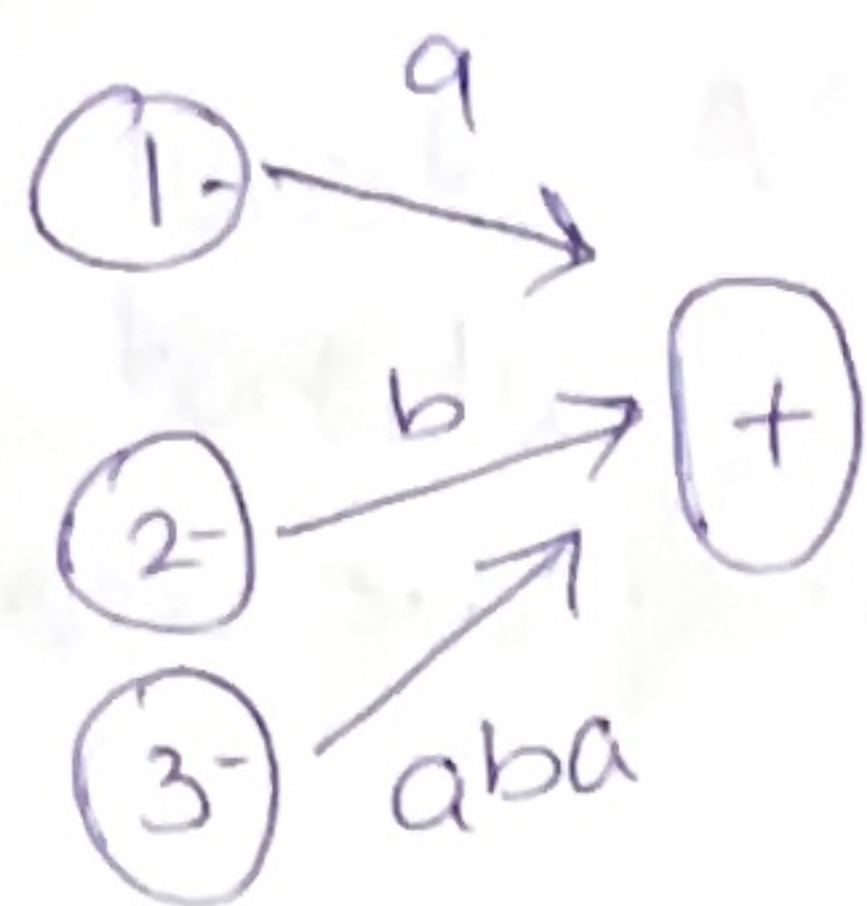


left → b, but no edge to follow ∴ the machine CRASHES

∴ STRING GOT REJECTED.

As we allowed tree edges ^, logically it is allowed for more than one state as shown:





Now real difference between

The two mentioned TGs.

→ in sense that all strings accepted by the first are also accepted by the other.

D/f lies b/w the no. of states.

~ x ~

RULE :-

FA is a TG, however

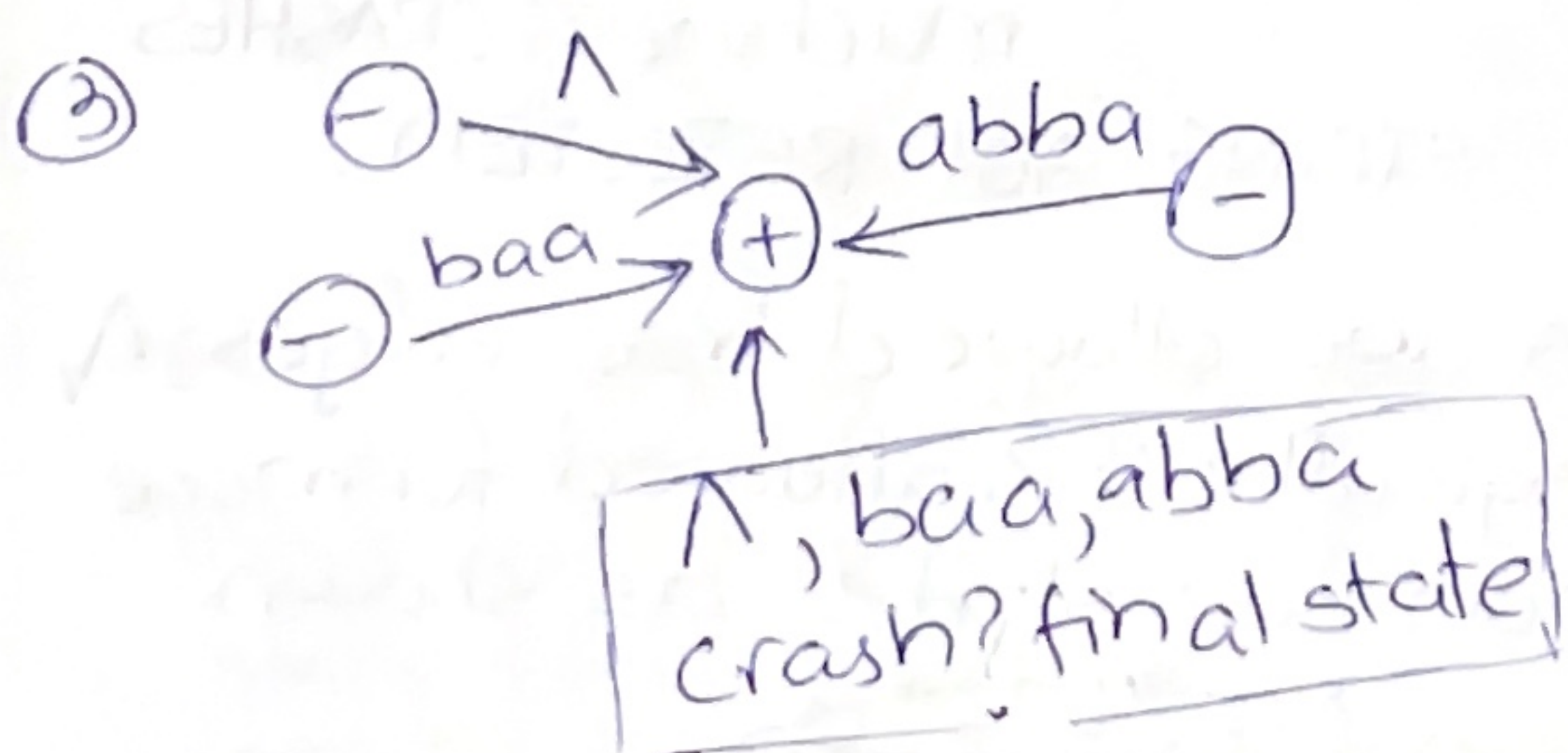
All TG do not satisfy the definition of FA.

~ x ~

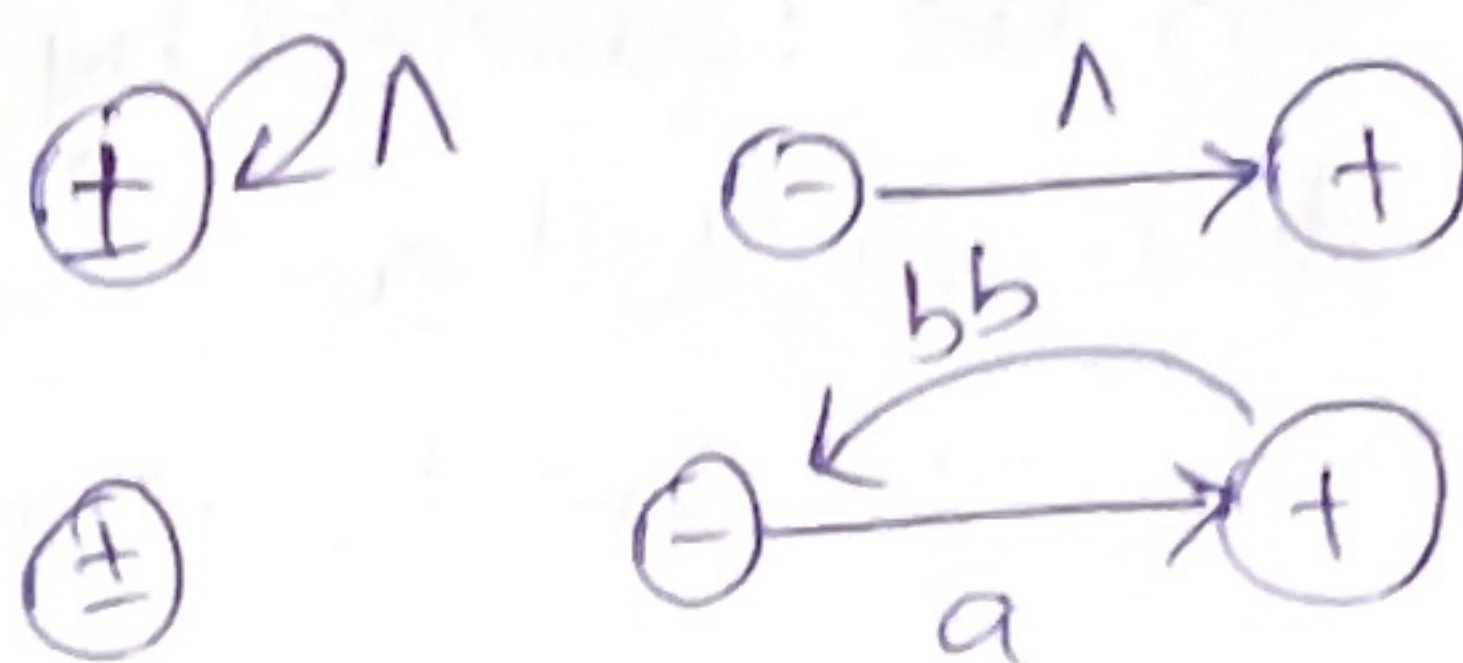
LOOKING AT TRANSITION GRAPHS

① $\ominus$ Accepts nothing.

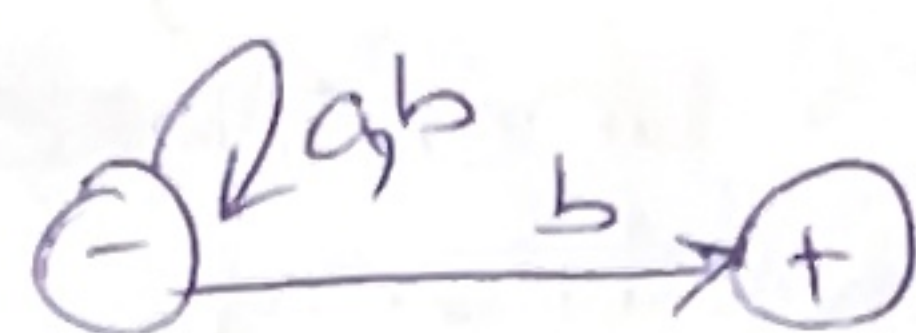
② $\oplus$ Accepts only strings (also true for FA).



EXAMPLE



EXAMPLE



- can read all the input letters and stay on the same state.

- when read 'b';
2 possibilities exist.
① stay there
② move to next state.

- it should be the last otherwise machine will crash, why?

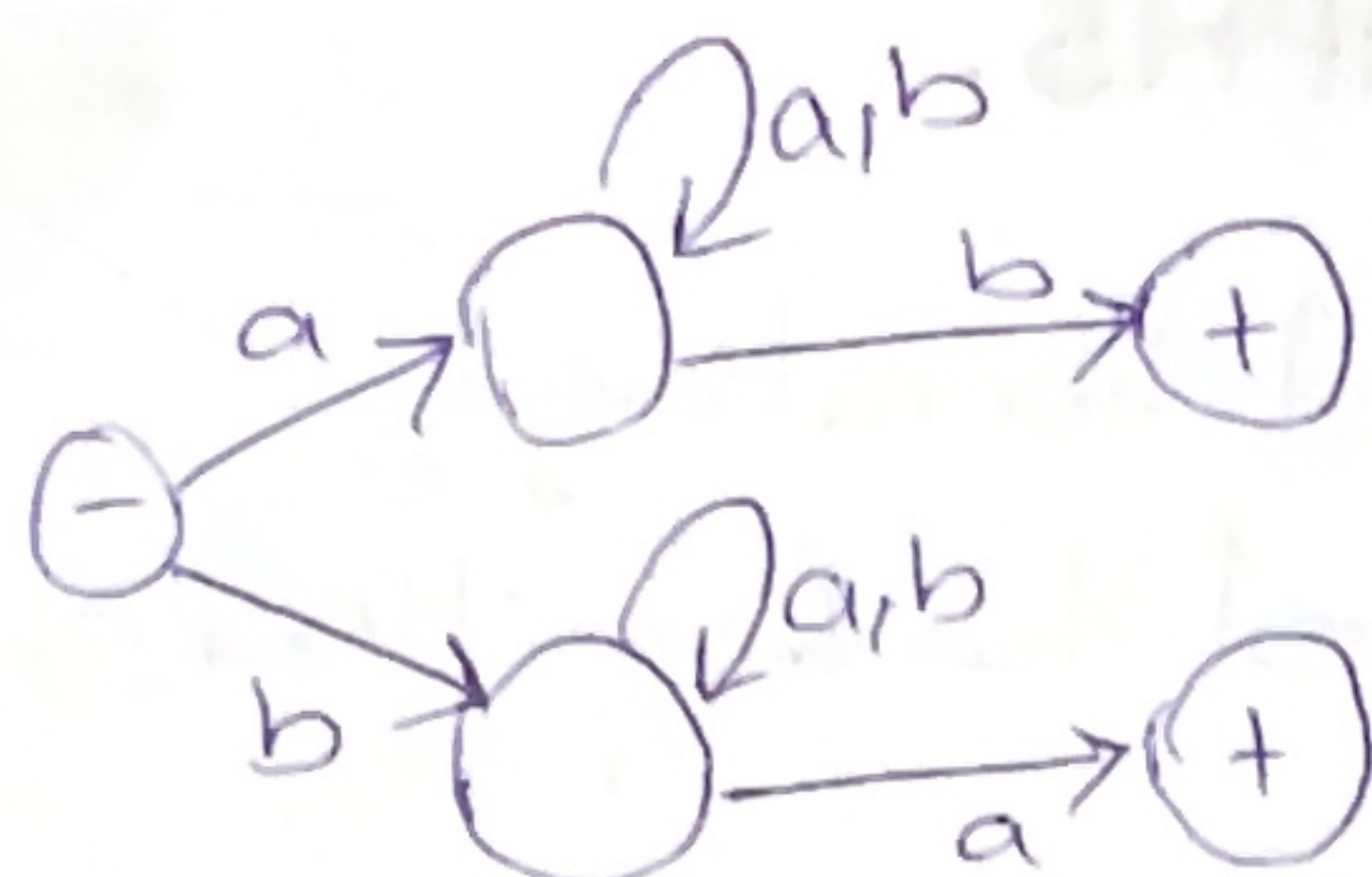
→ no outgoing edge after reaching the final state.

Language accepted by the TG is thus all words ending with the letter b.

RE $\Rightarrow (a+b)^*b$.

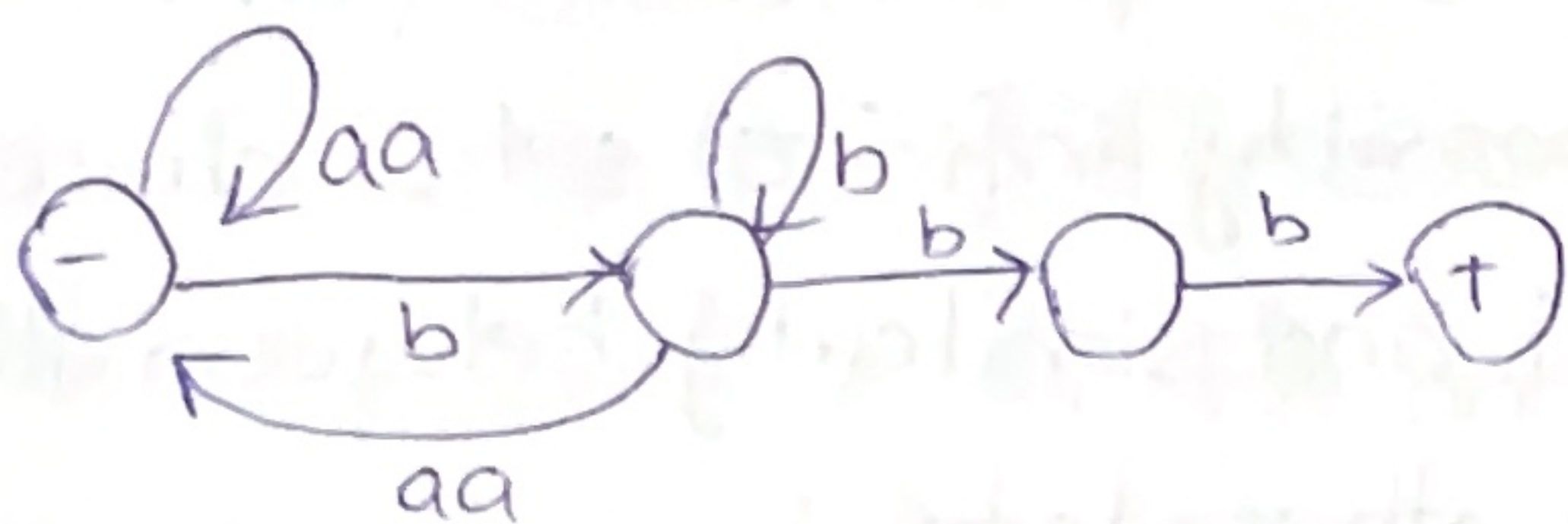
Pg no. 7:

EXAMPLE



Language that accepts all words ending with different letters.

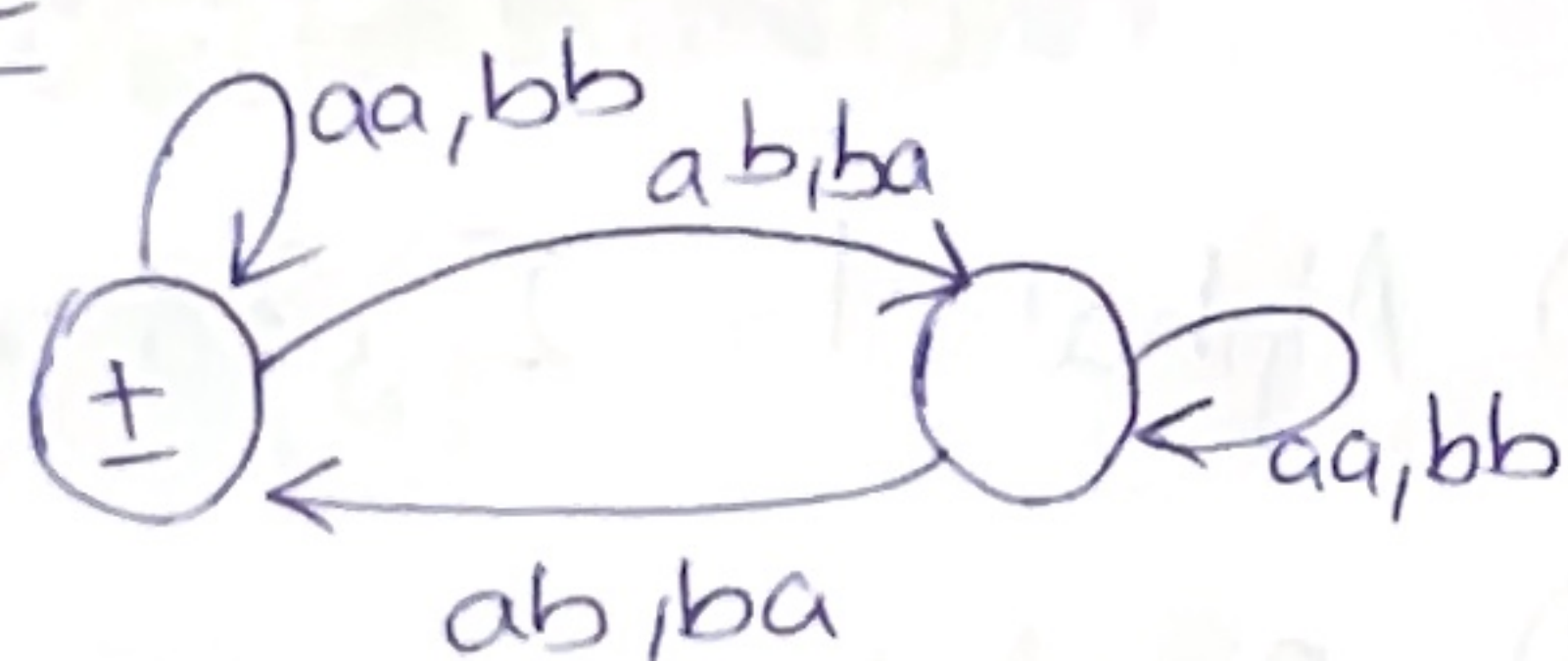
Example



Language that accepts all words in which a's occur only in even clumps and then ends with three or more 'b's'.

* Note :- No edge to read a single 'a'

Example



- Every edge has a pair of letters.

- This means if we want the string to be acceptable it must be even; read & processed in 2(s)

i.e Even no. of letters.

⇒ Even - Even language.

Consider traversing the following in this machine

- aa

- abba

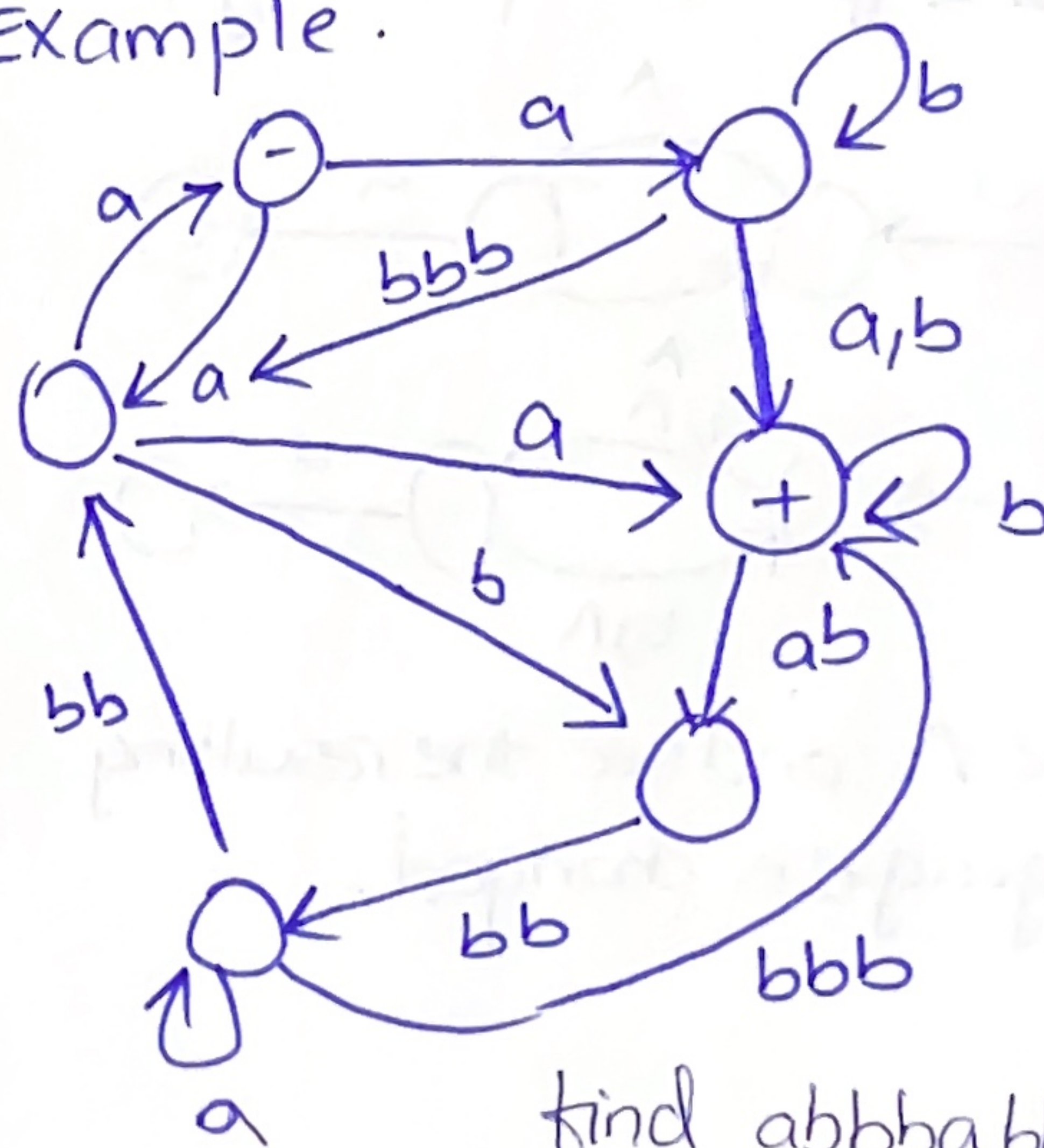
- abbab.

→ Note even no. of letters.

PROBLEMS WITH TRANSITION GRAPHS:-

So many possibilities of ways / possible ways of grouping the letter of the input string that we must examine many possibilities before we know whether a given string is accepted or rejected.

Example.



find abbba bbbabba is accepted or not?

Yes! By 3 paths / way.

Note: Λ / lambda edges allow infinite no. of ways of grouping letters.



Many paths of infinite length. look into, Λ -loop edges.

Why Λ is allowed? When the value is nil.

makes it simple

- any edges, any where with any labels.

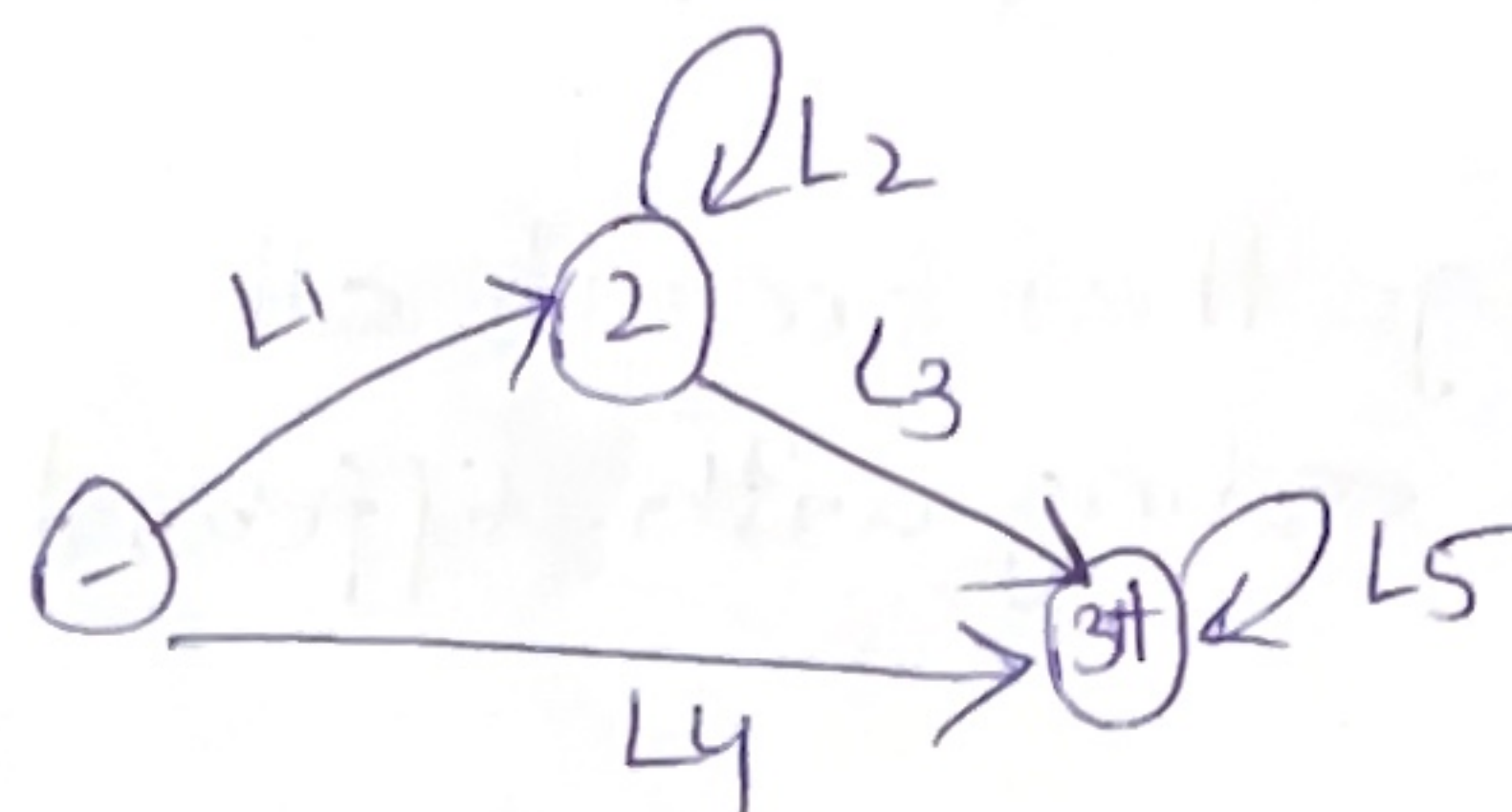
Question: is Λ loop the only way to get infinite path out of a finite input string?



erase Λ and see the resulting language is changed.

GENERALIZED TRANSITION GRAPHS.

Advantage of liberating state - to - state transition.



we can travel from state 1 to state 2 by reading any (of course finite) word from (possibly infinite) set of choices $L1$ and similarly between all the other states.

DEFINITION:-

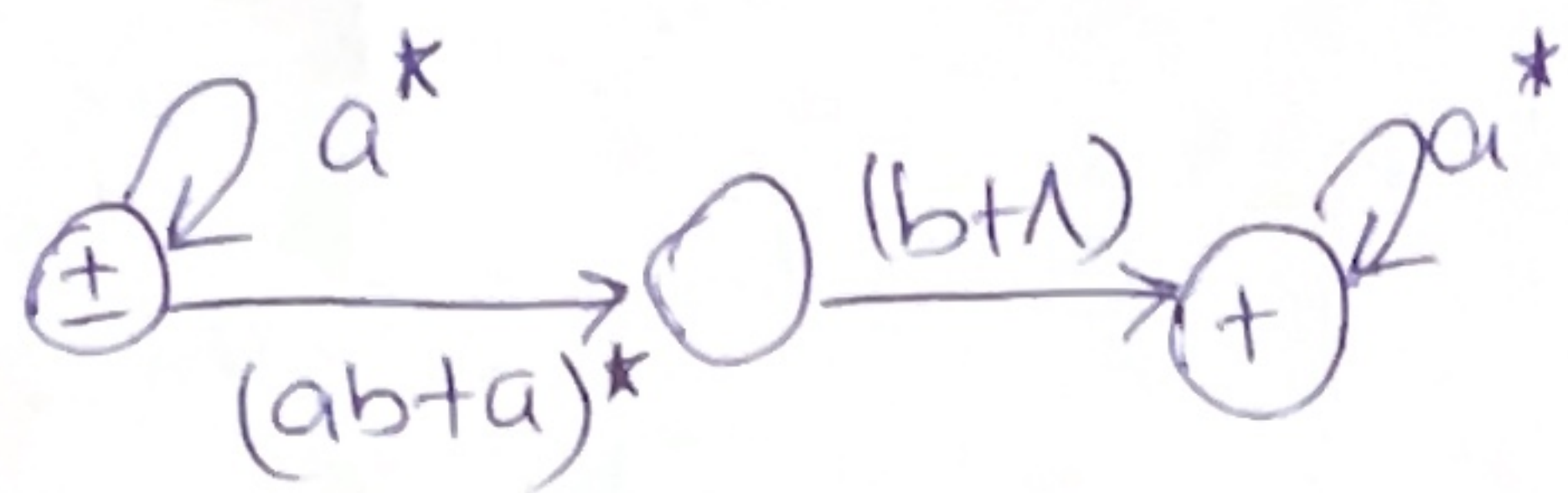
We call the approach as generalized transition graphs or GTG

① a finite set of states one is start / initial state and other is the final state.

② Alphabets Σ of input letters.

③ Directed edges connecting some pair of states each labelled with R.E.

EXAMPLE :-



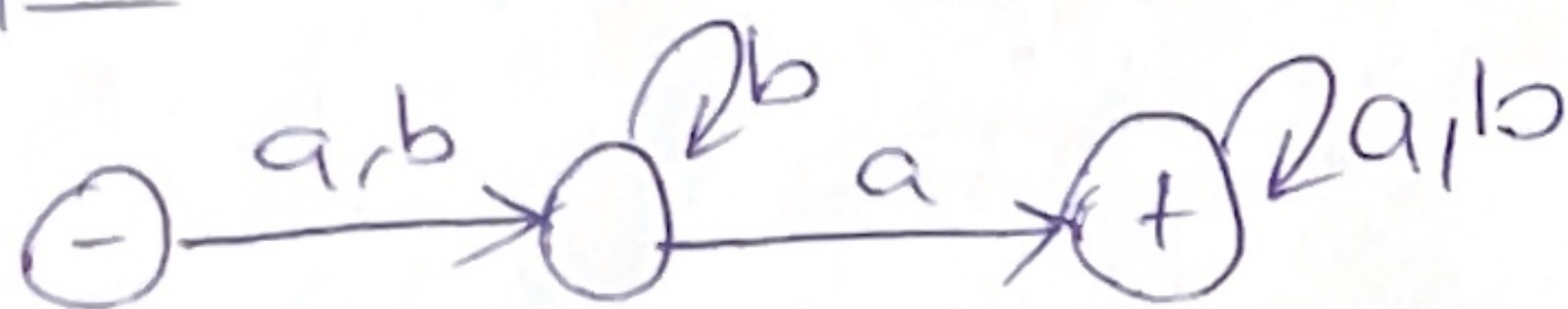
Accepts all the strings without double 'b'

Notice Λ -edge from start to middle state for the word 'b'.

Remember:

There is no difference b/w $\emptyset$ and $*$ in FA and TGs (respectively).

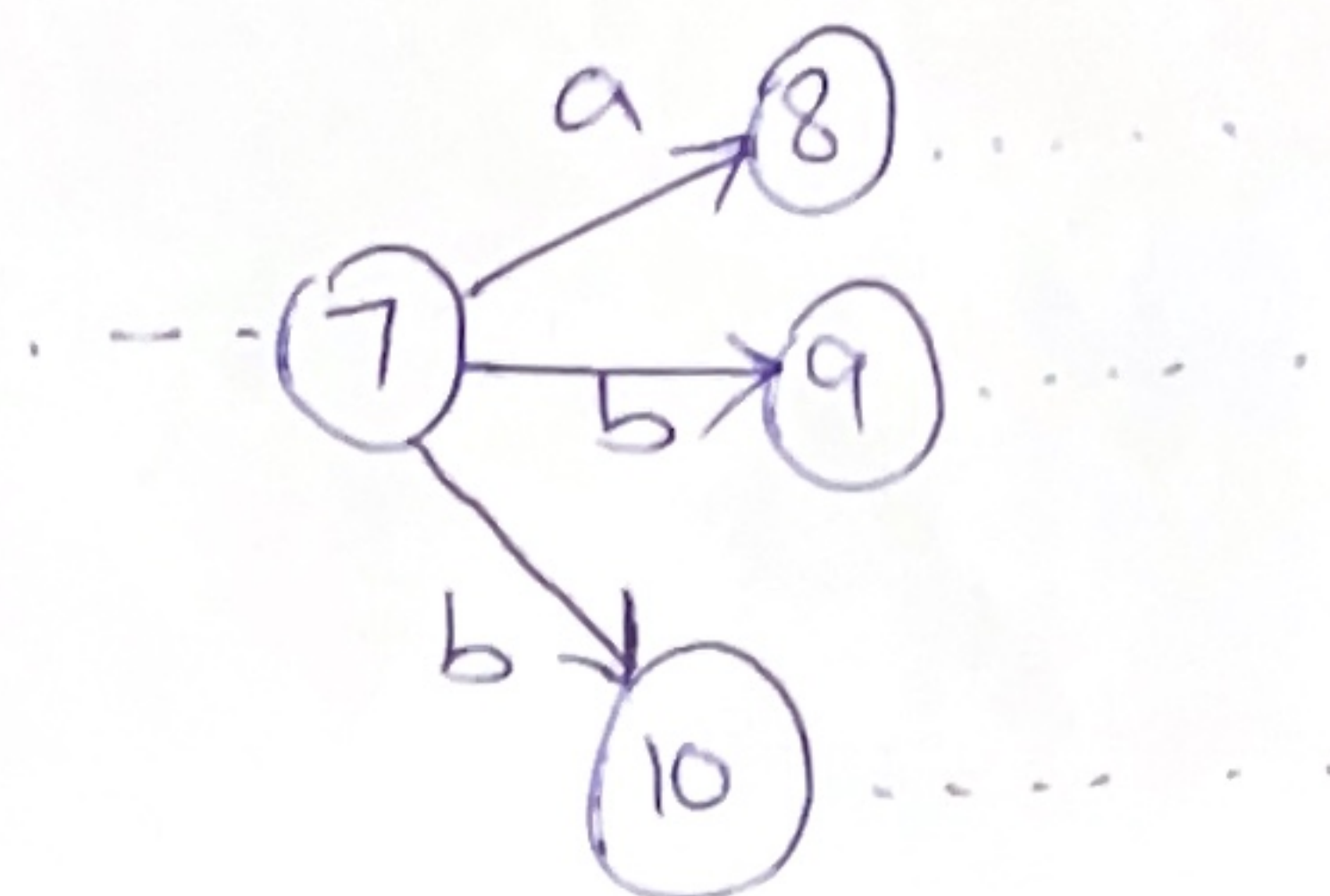
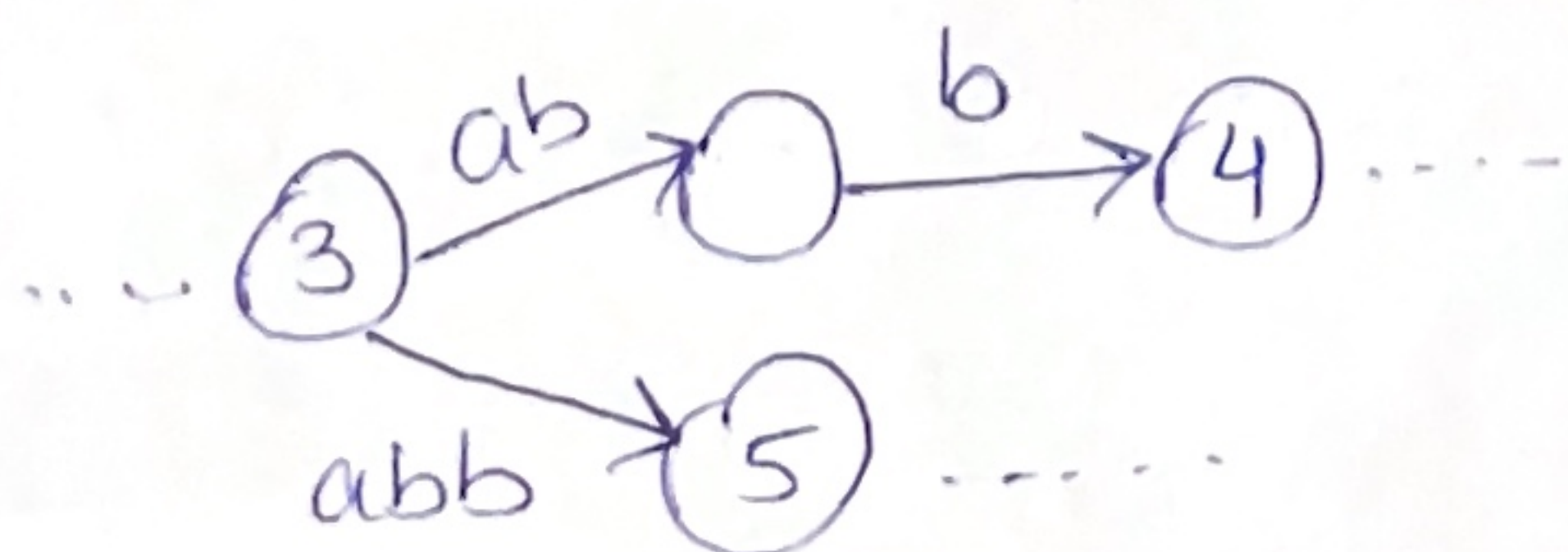
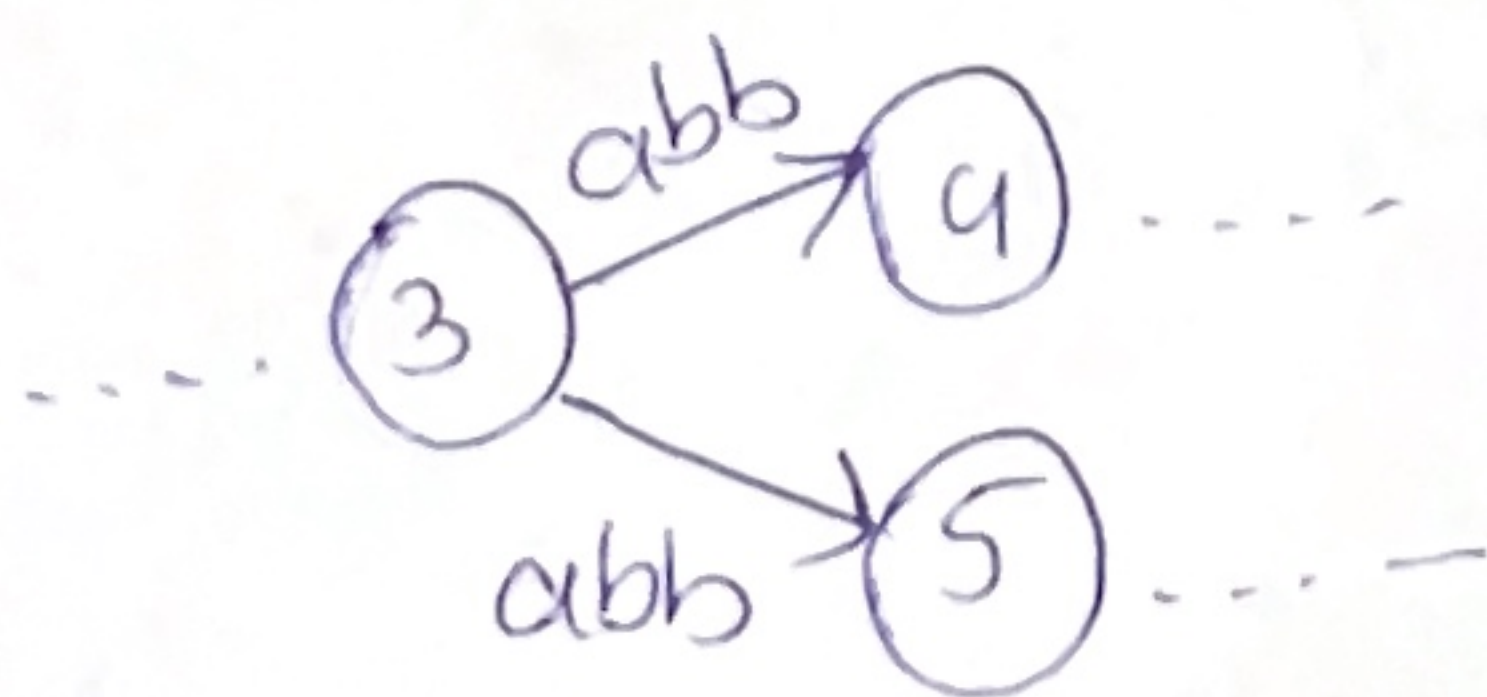
Compare



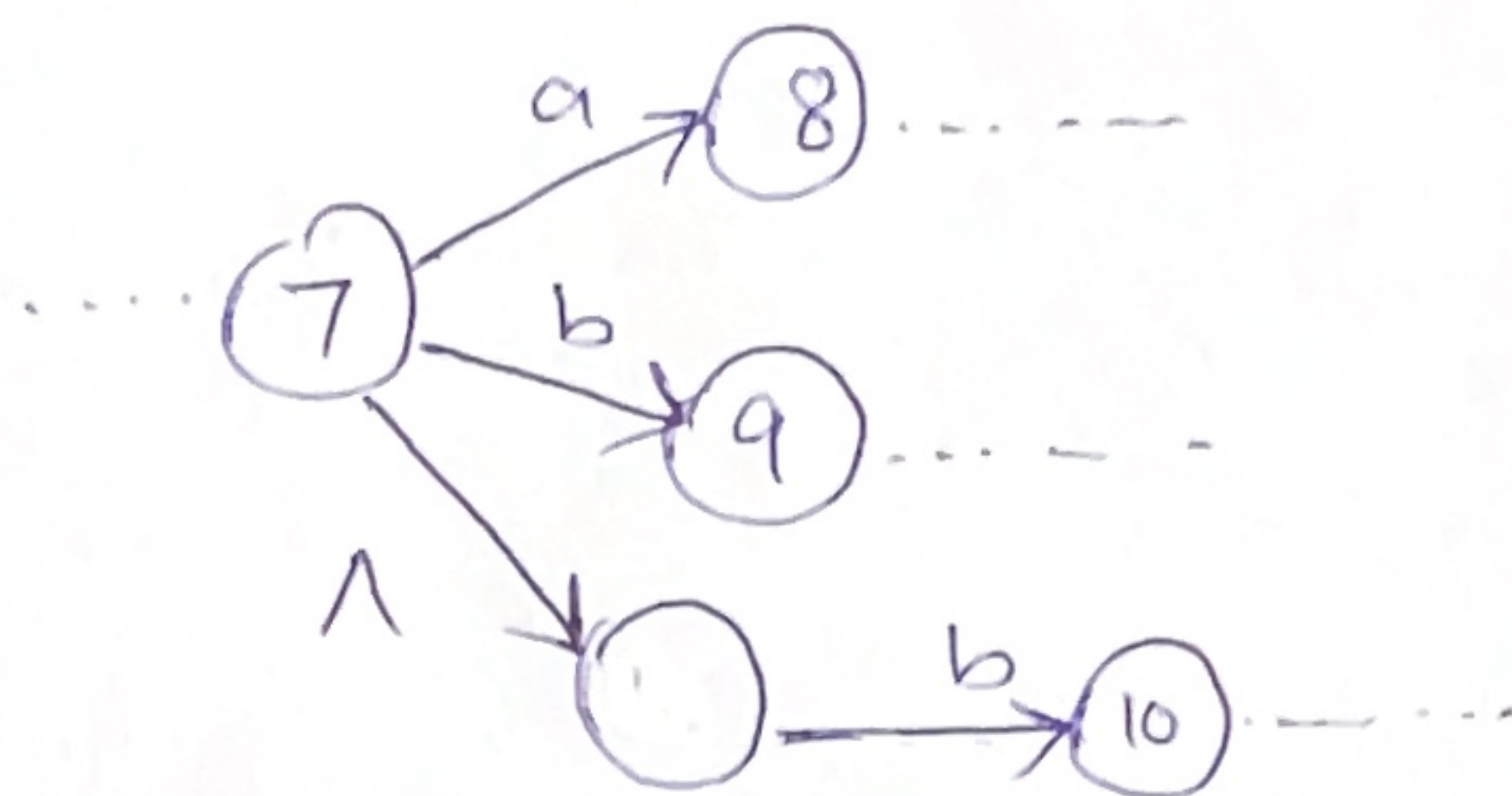
AND



NON-DETERMINISM.



equals;



END!

**THEORY OF COMPUTATION
AND FINITE AUTOMATA
- CS224.**